

MPC for reaction wheel pendulum with motor voltage limits

Christopher Xu
ECE594T Spring 2026

I. INTRODUCTION

Monopedal jumping robots can achieve rapid locomotion over complex terrain and are used to study impulsive actuation and dynamic legged control [1] [2]. Jumping can be an especially effective form of locomotion in planetary exploration missions where lower gravity allows for farther jumps and thinner atmosphere makes conventional flight difficult [3]. To steer, a monopedal robot may reorient itself to point in the desired direction prior to the jump, often with inertial flywheels.

Authors of [4] designed a 2D balance controller capable of tracking predefined motion references in body pitch. Their controller consists of two to four feedback gains found through pole placement. Additionally, they pass the reference through a time-reversed low-pass filter to account for non-minimum phase behavior, resulting in controller actions that lean in anticipation of future movements.

However, a linear controller does not account for actuator saturation limits, so at larger orientation errors sustained for longer periods of time or at high initial wheel velocity, the reaction wheels may reach the maximum speed of the actuator without righting the robot. For a given voltage, the torque of a DC motor decreases linearly with its velocity such that it reaches zero torque at some maximum free speed. Additionally, magnetic saturation in the iron core and driver current limits create bounds on the torque. This results in a parallelogram-shaped torque-speed operating envelope [5].

Model predictive control can reason about the speed-dependent torque limits and allow for effective pitch control in cases with high initial reaction wheel speed, lower inertia reaction wheels, larger orientation error, or less powerful actuators. If control can be accomplished with smaller flywheels and weaker motors, the robot can be lighter and execute farther jumps.

II. METHODS:

A 2D monopedal robot balancing with a flywheel can be modeled as a reaction wheel pendulum, whose equations of motion are given by [4]:

$$\ddot{q}_1 = \frac{Mgc \sin(q_1) - \tau}{I_1}$$

$$\ddot{q}_2 = -\frac{Mgc \sin(q_1)}{J_{\text{pend}}} + \frac{\tau(I_1 + I_2)}{I_2 I_1}$$

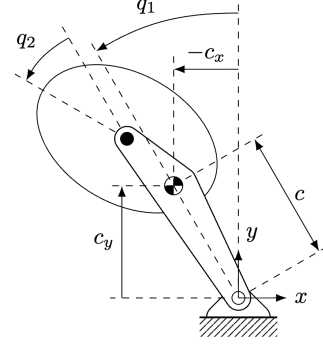


Fig. 1. The reaction wheel pendulum, as drawn by [4].

where M is the total mass, q_1 is the angle of the body, q_2 is angle of the flywheel, c is the distance of the center of mass to the pivot, I_1 is the inertia of the body, and I_2 is the inertia of the flywheel.

Since the angle of the flywheel does not matter, the state is chosen to be $\mathbf{x} = [q_1, \dot{q}_1, \dot{q}_2]^T$, then the system is linearized about the balanced state ($q_1 = \dot{q}_1 = \dot{q}_2 = 0$) and discretized.

To be able to run the MPC on a microcontroller, TinyMPC [6] was selected as an interface for testing convex MPC problems in Python and generating lightweight C++ code. TinyMPC defines the quadratic program:

$$\begin{aligned} & \text{minimize} && \sum_{l=k}^{k+K-1} g(x, u) = (x - x_{\text{ref}})^T Q (x - x_{\text{ref}}) + u^T R u \\ & \text{subject to} && x(0) = x_0, \\ & && x(l+1) = \mathbf{A}x(l) + \mathbf{B}u(l), \quad l = k, \dots, k+K-1, \\ & && \mathbf{A}_{cx}x(l) \leq b_{cx} \\ & && \mathbf{A}_{cu}u(l) \leq b_{cu} \\ & && x_{\min} \leq x(l) \leq x_{\max} \\ & && u_{\min} \leq u(l) \leq u_{\max} \end{aligned}$$

where $\mathbf{A}$ and $\mathbf{B}$ are the discrete-time linearized dynamics, and $\mathbf{A}_{cx}$, $\mathbf{A}_{cu}$, b_{cx} , b_{cu} are linear constraint matrices on the state and inputs.

The motor torque limits are

$$-\tau_{\text{stall}}\left(1 + \frac{\dot{q}_2}{\omega_{\text{free}}}\right) \leq \tau \leq \tau_{\text{stall}}\left(1 - \frac{\dot{q}_2}{\omega_{\text{free}}}\right)$$

$$-\tau_{\max} \leq \tau \leq \tau_{\max}$$

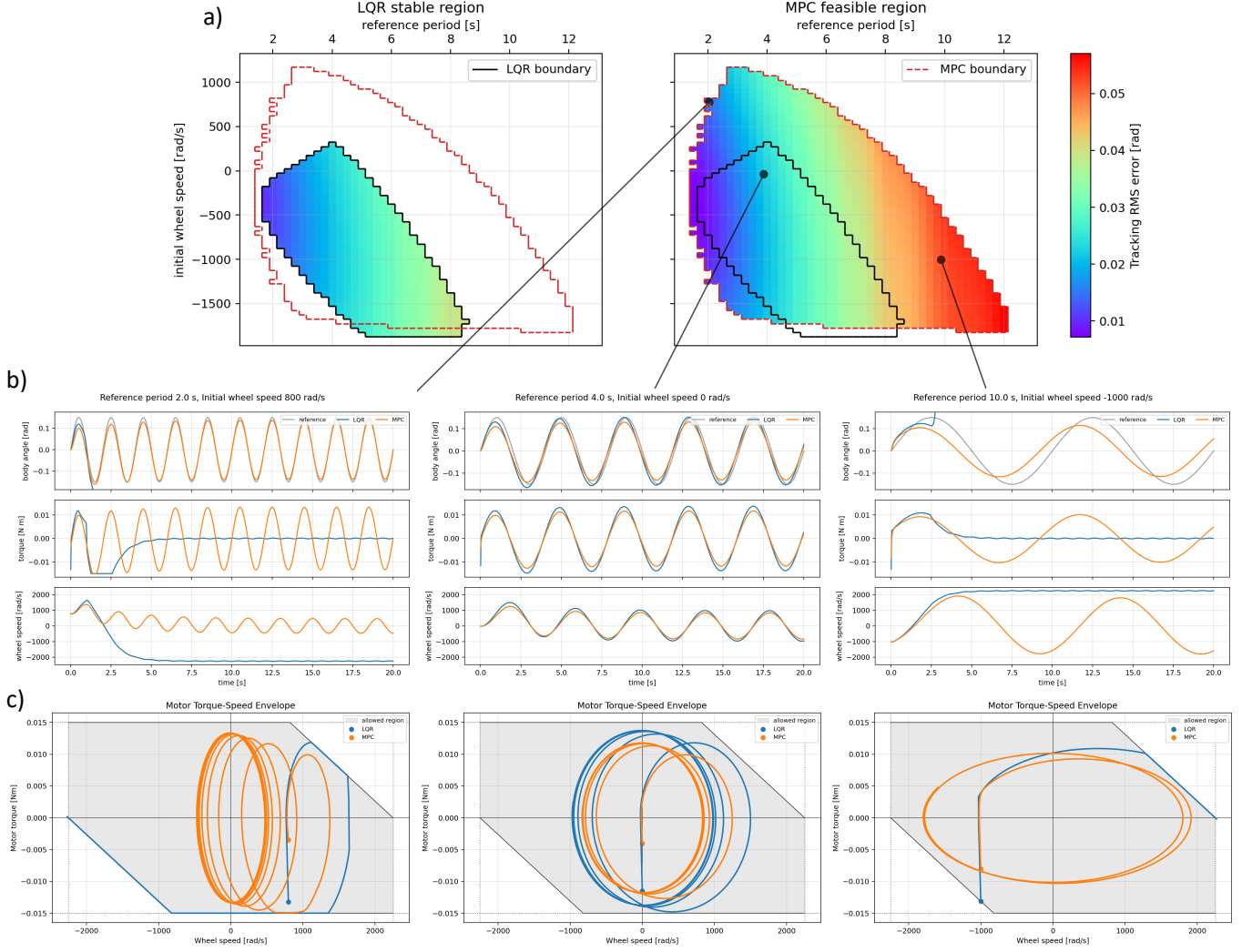


Fig. 2. **a)** The LQR and MPC controllers are swept in simulation with varying initial wheel speed and reference sinusoid period. Other initial states are set to zero and weight matrices are kept constant. The LQR is unstable outside the solid black outlined region, and the MPC fails to solve outside the dashed red outlined region. Root mean square tracking error is indicated by color when stable/feasible.

b) Simulated body angle, torque, and wheel speed trajectories are plotted for three cases. On the left, a fast reference and high positive initial wheel velocity causes the LQR to overcompensate on the first trough and results in saturating the wheel with negative velocity, while MPC closely tracks the reference after a few periods, gradually decreasing wheel speed. In the middle, both LQR and MPC can track the moderate reference with no initial wheel speed. On the right, a slow reference causes the LQR to saturate the wheel with positive velocity, while MPC is more conservative and stays stable, although with poor tracking performance.

c) The motor operating point along each trajectory is plotted inside the theoretical torque speed envelope of the motor including torque saturation at ± 0.015 Nm and voltage limits.

where $\tau_{stall} = \frac{V}{RK_T}$, τ_{max} is the maximum torque, V is the maximum voltage, R is the phase resistance, K_T is the motor torque constant in Newton-meters/amp, and $\omega_{free} = \frac{V}{K_T}$ is the unloaded speed of the motor.

The speed-dependent constraint on motor torque would require a constraint that compares both u and x . This is not possible using the above formulation, so instead I augment the torque τ to the state and define u to be its derivative: $\dot{\tau} = u$. Now, the constraint can be implemented linearly on just the state:

$$\mathbf{x} = [q_1, \dot{q}_1, \dot{q}_2, \tau]^T$$

$$A_{cx} = \begin{bmatrix} 0 & 0 & \frac{\tau_{stall}}{\omega_{free}} & 1 \\ 0 & 0 & -\frac{\omega_{free}}{\tau_{stall}} & -1 \end{bmatrix}, b_{cx} = \begin{bmatrix} \tau_{stall} \\ \tau_{stall} \end{bmatrix}$$

To test the performance of the system, the full nonlinear dynamics are integrated with RK4 at $dt=0.02$. A sinusoidal reference $x_{ref}(t)$ was pre-generated and during each time step, the quadratic program is solved using TinyMPC and the torque is extracted from the next predicted state to apply to the system. A horizon length of 30 steps (0.6 seconds) is used.

III. RESULTS

As a baseline, the MPC described above is compared to a standard infinite-horizon LQR controller on the states $\mathbf{x} = [q_1, \dot{q}_1, \dot{q}_2]^T$ with the same Q and R gains:

$$Q_{\text{LQR}} = \begin{bmatrix} 1000 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 0 & 10^{-6} \end{bmatrix}, \quad R_{\text{LQR}} = [1]$$

$$Q_{\text{MPC}} = \begin{bmatrix} 1000 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 \\ 0 & 0 & 10^{-6} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_{\text{MPC}} = [1]$$

Simulation results are plotted in Fig.2, with constants shown in Table I.

body mass (g)	100
flywheel mass (g)	20
body height (mm)	150
flywheel radius (mm)	30
motor K_T (N*mm/A)	2.22
motor voltage (V)	5
motor phase resistance (m Ω)	470

TABLE I

For this arbitrary configuration, the MPC is feasible across more initial wheel velocities and reference periods than the LQR is stable. At high positive initial wheel speeds, the controller must budget its torque input so that it does not saturate the wheel velocity on the first reference peak. Slower references are difficult to track well because the desired body angle remains away from equilibrium for an extended period of time, increasing likelihood of wheel saturation. At high negative initial wheel speeds, there is more angular momentum to work with when the reference angle increases at the start of simulation.

Fig. 2c) shows that the MPC avoids constraints on motor torque as much as possible while the LQR is agnostic and gets clipped. The LQR results in higher torques overall and is overly aggressive when the actuators are at risk of saturation. In certain cases, the MPC reduces actuation at the start to trade off some tracking performance in order to obtain a more favorable wheel velocity that helps future tracking.

After verifying in python simulation, the MPC controller was converted into C++ code and benchmarked on an ESP32-S3 microcontroller. Using the same horizon length and gains, the embedded version runs with a average time per iteration of 670 microseconds, which is well below the 0.02s control rate. Simulation and embedded code is public at <https://github.com/qwertpas/reaction-wheel-mpc>.

IV. DISCUSSION AND NEXT STEPS

The MPC controller was originally intended to account for motor velocity saturation for inertial midair reorientation as a replacement for the PD controller on projected leg orientation implemented in [3]. However, my initial simulations showed that an MPC that accounts for motor limits performed about as well as an LQR controller that is clipped to obey motor

torque limits. The only difference was that the MPC reacted slightly earlier due to its preview horizon, though this can be replicated in a linear controller using the time-reversed low-pass filter in [4]. I thought that the disappointing difference in performance was due to the simplicity of the system, as a linearized system reorienting in midair is just a double integrator. Therefore, the controller was applied and compared to LQR on a slightly more complicated non-minimum phase system to take advantage of the predictive horizon.

The tradeoff between MPC horizon length and control rate can be explored further in this application, as the microcontroller solves each iteration faster than expected and likely can handle much longer horizons or higher control rates, which may lead to better tracking performance on more complicated references. Additionally, a potential next step might be to extend the MPC controller to balancing along two axes using two orthogonal flywheels. The full dynamics have cross-coupling terms but the linearized system would decouple the two axes, so it is unclear whether a linear MPC with constant A and B would be as effective.

REFERENCES

- [1] M. H. Raibert, H. B. Brown Jr, and M. Chepponis, "Experiments in balance with a 3d one-legged hopping machine," *The International Journal of Robotics Research*, vol. 3, no. 2, pp. 75–92, 1984.
- [2] D. W. Haldane, J. K. Yim, and R. S. Fearing, "Repetitive extreme-acceleration (14-g) spatial jumping with salto-1p," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 3345–3351.
- [3] N. R. Wagner and J. K. Yim, "Underactuated multimodal jumping robot for extraterrestrial exploration," *arXiv preprint arXiv:2603.06525*, 2026.
- [4] J. J. Driessen, A. E. Gkikakis, R. Featherstone, and B. R. P. Singh, "Experimental demonstration of high-performance robotic balancing," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 9459–9465.
- [5] Y. Sim, G. Colin, and J. Ramos, "Control- task-aware optimal design of actuation system for legged robots using binary integer linear programming," in *2023 IEEE-RAS 22nd International Conference on Humanoid Robots (Humanoids)*, 2023, pp. 1–8.
- [6] K. Nguyen, S. Schoedel, A. Alavilli, B. Plancher, and Z. Manchester, "Tinympc: Model-predictive control on resource-constrained microcontrollers," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.